CCDV-F Practice Questions

58 original questions: the first 40 allocated roughly proportional to domain weight, plus 18 additional questions (41–58) drawn from deeper Partner Academy course content in Domains 1, 2, 4, 7, and 8. Answers and explanations follow each question. These are original questions written from the public exam blueprint, official docs, and paraphrased course material — not reproductions of actual (confidential) exam items.

Domain 1: Agents and Workflows

1. A team can map out every step their document-processing task requires, in a fixed order, with no branching based on content. What should they build?

- **A.** An autonomous agent with broad tool access
- **B.** A workflow with a predefined code path
- **C.** An orchestrator-workers pattern
- **D.** A manager/supervisor hierarchy

> **Answer: B.** When the exact steps are known and fixed in advance, a workflow gives predictability and consistency that an open-ended agent doesn't need to trade away.

2. A central LLM breaks an unpredictable research task into subtasks whose number and shape depend entirely on what it finds, delegating each to a worker. Which pattern is this?

- **A.** Prompt chaining
- **B.** Routing
- **C.** Orchestrator-workers
- **D.** Evaluator-optimizer

> **Answer: C.** Orchestrator-workers is defined by dynamic, input-dependent decomposition — the orchestrator can't know the subtask structure ahead of time.

3. Why does using a subagent for a large file-exploration task keep the main agent's context window smaller than doing the exploration inline?

- **A.** Subagents use a different, larger context window
- **B.** Only the subagent's final summary returns to the parent; intermediate tool calls stay isolated
- **C.** Subagents automatically compress all tool output
- **D.** Subagents don't have a context window

> **Answer: B.** A subagent starts fresh and its parent only receives its final response as a tool result — the dozens of file reads it performed never enter the parent's context.

4. You need to guarantee a specific Bash command is blocked before it executes, regardless of what permission mode the session is running in, including `bypassPermissions`. What should you use?

- **A.** A more restrictive system prompt
- **B.** A `PreToolUse` hook that denies the call
- **C.** `disallowed_tools` in the session options
- **D.** A subagent with restricted tools

> **Answer: B.** A `PreToolUse` hook enforces even under `bypassPermissions` — it's a separate, earlier gate than the permission-mode decision, and it's a deterministic code-level guarantee rather than a prompt-level suggestion.

5. A team wants to run their agent entirely on their own infrastructure so nothing leaves their environment unless they explicitly send it, and they're willing to handle server provisioning and OAuth themselves. Which deployment model fits?

- **A.** Managed Agents (Anthropic-hosted)
- **B.** The Agent SDK, self-hosted
- **C.** Self-hosted sandboxes
- **D.** Claude Desktop

> **Answer: B.** Self-hosted Agent SDK deployment runs entirely on your infrastructure; you own provisioning, OAuth, secrets, and retries in exchange for full control and data staying in your environment.

6. Which third-party agentic framework is most associated with explicit, graph-based control over every step of a complex, stateful multi-agent workflow?

- **A.** Claude Agent SDK
- **B.** LangGraph
- **C.** Strands Agents
- **D.** PydanticAI

> **Answer: B.** LangGraph's defining characteristic is treating agent steps as explicit nodes in a directed graph, giving fine-grained control over state and transitions.

Domain 2: Applications and Integration

7. A developer needs to process 10,000 documents overnight for a non-urgent report, and cost is the primary concern. What should they use?

- **A.** Synchronous Messages API calls in parallel
- **B.** The Message Batches API
- **C.** Lower `max_tokens` on synchronous calls
- **D.** Switch to the smallest model regardless of quality

> **Answer: B.** Latency-tolerant, high-volume, cost-sensitive, no user waiting — exactly the Message Batches API's design point (up to 24h, lower per-token cost).

8. A conversation accumulates several images over many turns, sent as `base64` blocks each time. What's the main downside, and what fixes it?

- **A.** Base64 images can't be analyzed accurately; switch to `url` sources
- **B.** The full image payload re-sends on every turn, compounding size/latency; upload once via the Files API and reference by `file_id`
- **C.** Base64 is deprecated; only `url` sources are supported
- **D.** There is no downside — base64 is always preferred

> **Answer: B.** Base64 image blocks are part of the message content and get resent with the full growing history every turn; Files API upload-once/reference-by-`file_id` avoids that compounding cost.

9. You're porting Claude API code from the first-party API to Google Cloud's Agent Platform. What are the two concrete API-shape differences you must account for?

- **A.** The API is completely different and must be rewritten
- **B.** The model is specified in the endpoint URL, and `anthropic_version` is a required body field
- **C.** Tool use isn't supported on Google Cloud
- **D.** Streaming isn't supported on Google Cloud

> **Answer: B.** The Google Cloud API is nearly identical to the Messages API with exactly these two differences — model in the URL, and a required `anthropic_version` field.

10. A regulated company needs Claude API traffic to route through a specific geographic region for compliance reasons, using Amazon Bedrock. What should they choose?

- **A.** A global endpoint
- **B.** A regional endpoint
- **C.** A multi-region endpoint (that's Google Cloud only)
- **D.** There's no way to control this on Bedrock

> **Answer: B.** Bedrock regional endpoints guarantee routing through a specific geography, unlike global endpoints which dynamically route for maximum availability.

11. Why should eval suites be part of a Claude application's CI pipeline rather than run manually before release?

- **A.** Evals are only useful during initial development
- **B.** A prompt, config, or model-version change can silently shift quality; CI-gated evals catch regressions the way unit tests catch code bugs
- **C.** CI doesn't support running evals
- **D.** Evals are too slow to run automatically

> **Answer: B.** Because model behavior can shift on a version bump and prompts drift as usage evolves, "maintain" is never really finished — CI-gated evals are what makes ongoing maintenance tractable instead of relying on manual spot-checks.

12. A team built and tuned their agent entirely inside Claude Code, relying heavily on CLAUDE.md for persistent instructions. They now port the same logic to a raw API integration. What breaks?

- **A.** Nothing — CLAUDE.md works identically over the API
- **B.** There's no CLAUDE.md over the API; every persistent instruction must be explicitly re-sent via the `system` parameter each request
- **C.** The API doesn't support system prompts at all
- **D.** CLAUDE.md is automatically converted to a system prompt by the SDK

> **Answer: B.** CLAUDE.md is a Claude Code–specific mechanism, re-injected automatically in that interface. The raw API has no equivalent — persistence must be built explicitly into your own `system` parameter handling.

13. What's the risk of a code review process that only looks at application logic and skips tool-schema and prompt diffs?

- **A.** None — schemas and prompts aren't part of the application
- **B.** A tool schema change can silently break every caller relying on the old shape, with no compiler to catch it

- **C.** Tool schemas can't actually change once deployed
- **D.** Prompts are automatically versioned separately from code review

> **Answer: B.** Schema and prompt changes are part of the application's contract; treating them as outside review scope means breaking changes ship without the safety net a compiler would normally provide for a typed API.

14. A Claude Code plugin's `plugin.json` declares two dependency plugins. What happens when a user installs it from a marketplace?

- **A.** The dependencies must be manually installed separately every time
- **B.** Installation can auto-install the declared dependencies, per the plugin.json/marketplace resolution rules
- **C.** Dependencies are ignored unless declared in marketplace.json only
- **D.** Plugins cannot declare dependencies

> **Answer: B.** Plugin install can auto-install dependency plugins declared in `plugin.json`, with marketplace-entry fields able to override or supplement that declaration.

15. Why pin a specific, dated model ID in a production deployment rather than always using the latest alias?

- **A.** Dated IDs are required by the Terms of Service
- **B.** Pinning protects production from an unannounced behavior shift on model upgrade; you control when to move to a newer pin, validated against your eval suite
- **C.** Only dated IDs support tool use
- **D.** Aliases are more expensive than dated IDs

> **Answer: B.** Every model ID is a pinned snapshot; the point of pinning explicitly (rather than tracking "latest") is that you decide when to absorb a behavior change, verified by evals rather than assumed safe.

16. A support ticket classifier prompt returns `"Billing"`, `"billing"`, and full sentences interchangeably across runs, breaking a downstream router expecting one fixed label. What's the most direct diagnosis?

- **A.** The model is malfunctioning; switch models
- **B.** The prompt is missing an output constraint — it never specified the exact form, label set, or "no other text" rule
- **C.** Temperature needs to be set to 0
- **D.** The prompt needs more few-shot examples and nothing else

> **Answer: B.** This is a classic wrong-shape failure, diagnosed as a missing output constraint — though the fix in practice often layers in a system-prompt contract and few-shot examples to show the exact label casing.

17. Which of the following is a genuine Systems Life Cycle concern specific to Claude applications, beyond standard SDLC practice?

- **A.** Version control isn't needed for prompts
- **B.** The "maintain" phase is never fully complete, because model behavior and prompt effectiveness can drift independent of any code change
- **C.** Claude applications don't require a design phase
- **D.** Testing is optional for LLM-based features

> **Answer: B.** Unlike traditional software with a fixed spec, a Claude application can degrade or shift in quality purely from model updates or usage-pattern drift, with no code change involved — which is why

> eval suites need to be built into the maintain phase from the start.

18. A team is deciding whether to route a workload through Amazon Bedrock or Microsoft Foundry. Which statement about model IDs is correct?

- **A.** Both platforms use identical model IDs to the first-party API
- **B.** Bedrock uses Bedrock-specific model IDs; Microsoft Foundry uses the same IDs as the first-party Claude API
- **C.** Neither platform lets you specify a model ID
- **D.** Model IDs are irrelevant to platform choice

> **Answer: B.** This is a concrete portability gotcha — Bedrock model IDs differ from the Claude API's own, while Foundry reuses the first-party IDs directly.

19. What's the primary reason to treat prompt changes with the same rigor (version control, review, eval validation) as code changes?

- **A.** Prompts are technically stored as code files
- **B.** A small wording change can measurably shift the model's output distribution, and nothing else (no compiler, no type system) will catch a regression before users do
- **C.** This is a legal requirement under the exam guide's NDA
- **D.** Prompts cannot be changed once an application ships

> **Answer: B.** Non-determinism and sensitivity to phrasing mean prompt regressions are silent unless caught by the same discipline (VCS, review, evals) applied to code.

Domain 3: Claude Code

20. A CI pipeline needs to run Claude Code with the exact same result on every machine, without picking up a teammate's personal hooks or an MCP server defined in the project's `.mcp.json`. What flag accomplishes this?

- **A.** `--continue`
- **B.** `--bare`
- **C.** `--output-format json`
- **D.** `--allowedTools`

> **Answer: B.** `--bare` skips auto-discovery of hooks, skills, plugins, MCP servers, auto memory, and CLAUDE.md, giving deterministic behavior independent of the host machine's local configuration.

21. A project has a `.claude/settings.local.json` with a permission `allow` rule, and the team's checked-in `.claude/settings.json` has a `deny` rule for the same tool. What happens?

- **A.** Local always wins outright, since it's higher in the precedence order
- **B.** Permissions accumulate across scopes rather than following strict override precedence — both rules apply
- **C.** Project always wins for permissions specifically
- **D.** This configuration is invalid and Claude Code will error

> **Answer: B.** Permissions are the documented exception to settings.json's normal override hierarchy — allow/deny/ask rules accumulate from every applicable scope instead of the highest scope simply winning.

Domain 4: Eval, Testing, and Debugging

22. An API call returns a 429 status. What should the client do?

- **A.** Treat it as a permanent failure and alert immediately
- **B.** Retry with exponential backoff, honoring the `retry-after` header
- **C.** Switch to a different model and retry once
- **D.** Reduce `max_tokens` and retry immediately with no delay

> **Answer: B.** `rate_limit_error` (429) is retryable — back off and honor the server's stated retry-after guidance rather than hammering the endpoint or treating it as fatal.

23. A response comes back with `stop_reason: "refusal"` despite a well-formed request and correct tool schema. Where should you look first?

- **A.** Your request-construction code for a bug
- **B.** What was actually asked for — the prompt/task itself, since the request layer was fine
- **C.** Your network configuration
- **D.** The API's rate limit settings

> **Answer: B.** A clean request that still gets refused points at the content of the ask, not the integration layer — this is a model-output-side signal, not an integration-layer bug.

Domain 5: Model Selection and Optimization

24. A team hardcodes "4 characters per token" to estimate cost across all their Claude-powered features, including ones recently migrated to Fable 5. What's wrong with this?

- **A.** Nothing — this ratio is fixed across all models
- **B.** The chars-per-token ratio depends on the model's tokenizer, which changes across generations (Fable 5's produces ~30% more tokens than pre-Opus-4.7 models for the same text)
- **C.** Token counting is only relevant for output, not input
- **D.** Cost is based on characters, not tokens

> **Answer: B.** Tokenizers differ by model generation; a fixed chars-per-token constant will systematically misestimate cost the moment you're mixing model generations.

25. A request's input already exceeds the model's context window before generation starts. What happens?

- **A.** The model truncates the input silently and proceeds
- **B.** The request is rejected with a validation error before generation begins
- **C.** The model returns a partial response with `model_context_window_exceeded`
- **D.** The request succeeds but the response is empty

> **Answer: B.** This is the "too big before it even starts" failure mode — a validation error up front, distinct from the "fits on input, hits the ceiling mid-generation" case (which returns a partial response with `model_context_window_exceeded`).

26. A test asserts that a Claude-generated summary exactly matches a fixed string. It fails intermittently even though the summaries are all correct. What's the best fix?

- **A.** Set `temperature` to 0 and assume determinism
- **B.** Assert on the property that must hold (required content present, structure parses) instead of exact text match
- **C.** Retry the test until it passes
- **D.** Switch to a smaller model for more predictable output

> **Answer: B.** Non-determinism means equally-correct answers can differ in wording; exact-string assertions are the wrong tool. Note also that `temperature: 0` makes output more repeatable, not identical — it wouldn't fully fix this either.

27. A workload needs the fastest, cheapest option, and an eval confirms the quality drop from Sonnet is acceptable for this specific task. Which tier fits, and what reasoning-mode caveat applies?

- **A.** Haiku 4.5 — and note it supports extended thinking, not adaptive thinking
- **B.** Opus 5 — no reasoning-mode caveat
- **C.** Fable 5 — always-on adaptive thinking
- **D.** Sonnet 5 — adaptive thinking

> **Answer: A.** Haiku 4.5 is the fastest/cheapest tier, and it's the odd one out on reasoning mode — it supports extended thinking (`thinking.type: "enabled"`), not the adaptive thinking that Fable/Opus/Sonnet 5 use.

28. A team sets `budget_tokens` to control reasoning depth on a newly-migrated Opus 5 integration and gets a 400 error. Why?

- **A.** `budget_tokens` requires a minimum account tier
- **B.** `budget_tokens` is deprecated and returns a 400 error on the newest model generations; `effort` levels replace it for adaptive thinking
- **C.** `budget_tokens` only works with streaming requests
- **D.** `budget_tokens` was renamed to `max_tokens`

> **Answer: B.** This is a direct, testable gotcha — the older token-budget control for reasoning depth doesn't carry over to the newest adaptive-thinking models; `effort` (low/medium/high/xhigh/max) is the current mechanism.

29. A workload calls the same cached prompt prefix roughly once every 20–30 minutes, with real value in keeping the cache warm across that gap. Which TTL should they choose, and what's the tradeoff?

- **A.** 5-minute TTL — free, no downside
- **B.** 1-hour TTL — survives the longer gap, at a higher cache-write price
- **C.** There's no TTL option; caching is always permanent
- **D.** TTL only applies to explicit breakpoints, not automatic caching

> **Answer: B.** The 1-hour TTL exists precisely for this bursty-but-infrequent pattern; it costs more to write but avoids a cold cache on every call given a 20–30 minute gap exceeds the 5-minute default.

Domain 6: Prompt and Context Engineering

30. A prompt has been re-worded five times and still doesn't produce the desired output shape. What should the developer do next?

- **A.** Add a sixth rewording with stronger language

- **B.** Stop and diagnose which of the four structural techniques (system prompt, few-shot, output constraint, XML/structure) is actually missing
- **C.** Increase the temperature to add variety
- **D.** Switch models and try again with the same prompt

> **Answer: B.** Repeated rewording without progress is the signature of skipping diagnosis — the fix is identifying the missing structural piece, not accumulating more phrasing.

31. A long-running agent's context is dominated by several large file reads whose content is no longer needed but could be re-fetched cheaply if required again. What's the appropriate context-management technique?

- **A.** Compaction (LLM-summarize the whole history)
- **B.** Pruning/clearing the old tool outputs
- **C.** Starting an entirely new session
- **D.** Reducing `max_tokens` on future requests

> **Answer: B.** Re-fetchable, low-ongoing-value tool output is exactly what pruning targets — cheaper and lossless compared to compaction, since the agent can just re-call the tool if needed.

32. Why does the exam guide's document-summarization example (indirect content) get delivered as untrusted `tool_result` content rather than injected into the system prompt, from a pure context-engineering (not security) standpoint?

- **A.** It's purely a security decision with no context-engineering rationale
- **B.** Isolating retrieved content in tool results also keeps the system prompt's instructions stable and uncontaminated as content is swapped in and out across turns
- **C.** System prompts can't contain any external content at all, by API design
- **D.** Tool results are cached differently from system prompts

> **Answer: B.** While the primary motivation is security (Domain 7), the same isolation also serves context hygiene — the system prompt stays a stable, cacheable instruction layer rather than being rewritten with every new document.

33. A tool that extracts structured fields from a support ticket needs a guaranteed-parseable response every time, even on inputs the team never tested. Which mechanism should they use, and why not just a strong prompt instruction?

- **A.** A stronger worded system prompt is sufficient on its own
- **B.** Structured outputs (`output_config.format`, `json_schema`) — a prompt-level instruction holds on tested cases and can slip on an untested edge case; schema constraints are enforced by the API on every token
- **C.** Increase `max_tokens` to avoid truncation
- **D.** Use few-shot examples exclusively, with no schema

> **Answer: B.** This is the core argument for structured outputs over prompt-only control: constrained decoding rules out invalid output at generation time rather than trusting the model to remember an instruction on inputs it wasn't tested against.

Domain 7: Security and Safety

34. A Claude-powered agent summarizes web pages submitted by end users. One page contains hidden text instructing the model to reveal its system prompt. What's the most effective mitigation?

- **A.** Raise the model's temperature so behavior is less predictable to attackers
- **B.** Treat retrieved page content as untrusted input, isolate it in tool results, and use guardrails/hooks so injected instructions can't trigger sensitive actions
- **C.** Add a system-prompt line asking users not to submit malicious pages
- **D.** Switch to a larger, more instruction-following model

> **Answer: B.** This is the indirect-injection defense pattern directly from the official exam guide's own sample question — isolation plus guardrails, not model tuning or a polite request.

35. Why JSON-encode a retrieved email body before including it in a tool result, rather than concatenating it as plain text?

- **A.** JSON encoding compresses the payload, reducing token cost
- **B.** JSON escaping provides unambiguous delimiters, so an attacker can't close a quote/tag and "break out" into an instruction context
- **C.** Plain text isn't supported in tool_result blocks
- **D.** JSON encoding is required by the Messages API schema

> **Answer: B.** The security value is structural: JSON's quoting makes embedded text unambiguously data, closing off the "breakout" technique adversarial content might otherwise use.

36. A `PreToolUse` hook is written to block `rm -rf` commands but returns exit code 1 when it detects one. The dangerous command still runs. What's wrong?

- **A.** `PreToolUse` hooks can't block Bash commands, only Edit
- **B.** Exit code 1 only produces a warning; exit code 2 is required to actually deny the tool call
- **C.** The hook needs to be registered under `PostToolUse` instead
- **D.** Hooks cannot inspect command arguments

> **Answer: B.** This is a specific, well-documented gotcha — exit 1 surfaces a warning without blocking anything; only exit 2 denies the call.

Domain 8: Tools and MCPs

37. A team needs Claude to call an internal inventory REST API, reusable across several Claude applications and maintained independently of any one app. What's the best approach?

- **A.** Hard-code the inventory logic into each application's system prompt
- **B.** Build an MCP server exposing the inventory operations as tools
- **C.** Paste the current inventory data into the context window on every request
- **D.** Rely on a built-in tool, since built-in tools can reach any internal API

> **Answer: B.** This is the exam guide's own Sample Question 3 pattern — MCP servers exist precisely for reusable, independently-maintained integrations shared across multiple client applications.

38. A subagent needs to examine code but must never modify files or run shell commands. How should its tool access be configured?

- **A.** Omit the `tools` field so it inherits everything, then rely on prompting to avoid edits
- **B.** Set `tools: ["Read", "Grep", "Glob"]` — a tool left out isn't available in the subagent's session at all
- **C.** Use `disallowedTools` on the main agent instead of the subagent
- **D.** There's no way to restrict a subagent's tools

> **Answer: B.** Explicitly listing only read-only tools is the correct restriction — an omitted tool isn't just discouraged, it's genuinely absent from that subagent's session, with no permission prompt or workaround.

39. Claude requests three independent `Read` tool calls and one `Edit` call in the same turn. How does the SDK typically execute them?

- **A.** All four run concurrently
- **B.** All four run sequentially, in the order requested
- **C.** The three `Read` calls can run concurrently; `Edit` runs sequentially (state-mutating tools avoid conflicts)
- **D.** Only one tool call is allowed per turn

> **Answer: C.** Read-only tools can run concurrently; tools that modify state run sequentially to avoid conflicting writes — this is the default parallel-execution behavior for built-in tools.

40. A team is deciding between a Skill and an MCP server for a workflow that needs to "look up the latest deployment status" from a live internal system. Which is correct, and why?

- **A.** A Skill alone is sufficient — Claude can infer live data through its instructions
- **B.** An MCP server is required to actually reach the live system; a Skill alone (a markdown file) cannot query external data — though a Skill can still add value on top by encoding how to use that data well
- **C.** Neither is appropriate; this requires a custom tool exclusively
- **D.** MCP and Skills are interchangeable for this use case

> **Answer: B.** MCP connects Claude to data; Skills teach Claude what to do with that data. A Skill describing "look up deployment status" still needs an MCP server (or custom tool) underneath it to actually fetch live data — the Skill alone can't reach outside its own text.

Additional questions (Domains 1, 2, 4, 7, 8 — deeper coverage)

41. A developer switches a session to `bypassPermissions` mode to stop constant prompts during a "routine" cleanup task. A glob pattern matches files in both the intended directory and an unrelated production config directory, and files are deleted with no confirmation. What specifically made this possible?

- **A.** `bypassPermissions` has a bug that will be patched
- **B.** `bypassPermissions` uniquely skips the protected-path guard that every other permission mode keeps, in addition to skipping all confirmation prompts
- **C.** The developer should have used `acceptEdits` instead, which would have prevented all deletions
- **D.** Glob patterns cannot match files outside the working directory in any mode

> **Answer: B.** `bypassPermissions` removes every safety checkpoint, including the protected-path guard the other modes retain — a broader-than-intended pattern match had nothing standing between it and the files it touched. (`acceptEdits` would not have fully prevented this either, since it auto-approves `rm` inside the working directory — the safer catch would have come from the script invocation prompting in `default` mode.)

42. A team is deciding whether a customer-facing configuration-editing agent needs a human-in-the-loop checkpoint before its `write_file` tool executes. The agent's own `validate_config` check already passed. What should determine whether a checkpoint is still required?

- **A.** If validation passes, no checkpoint is needed — the agent already confirmed correctness
- **B.** Whether the write is technically reversible via version control
- **C.** What the worst outcome is if this specific write runs without a human check — validation confirms a value is well-formed, not that nothing downstream depends on the old value
- **D.** Checkpoints are only needed for delete operations, never for writes

> **Answer: C.** This is the exact incident pattern from the material: `validate_config` passing meant the new value was schema-valid, not that no downstream system depended on the old value. The worst-case-outcome question, not validation status, determines whether a gate belongs before an irreversible action.

43. An agent performed well in development, run as continuous 10–15 turn sessions. In production, the same total work is spread across many separate, shorter sessions per day, with prior history injected at each session's start. By the fourth session, the agent starts returning incomplete results. What's the most likely cause?

- **A.** The model's capability degraded after several sessions
- **B.** Injected history accumulated across sessions and crowded out context before the agent could complete useful work — development and production used different session *shapes*, and in-context memory handles them differently
- **C.** The tool schemas are malformed
- **D.** This is expected token-budget behavior with no fix available

> **Answer: B.** This is a session-shape mismatch, not a capability or schema issue — one long session vs. many short accumulating ones. The fix is externalizing history to storage and injecting only the relevant subset at session start, ideally decided before deployment rather than as a production hotfix.

44. A workload needs long-running execution (measured in hours), and the team would strongly prefer not to build or secure an execution sandbox themselves. However, the workload also processes PHI under an existing HIPAA agreement. Which agent deployment path fits?

- **A.** Claude Managed Agents — it's designed exactly for long-running, sandboxed execution
- **B.** The Agent SDK or a raw Messages API loop on a BAA-covered configuration — Managed Agents' server-side stateful sessions are not currently HIPAA-BAA eligible, regardless of operational fit
- **C.** Either path works equally well; PHI status doesn't affect this decision
- **D.** No Claude deployment path can handle PHI workloads

> **Answer: B.** The governing constraint (PHI/BAA eligibility) overrides the operational preference. Managed Agents would otherwise be the natural fit for a long-running, sandbox-avoidant workload, but its server-side session storage currently rules it out for BAA-covered PHI regardless of how well it fits everything else.

45. Anthropic's own multi-agent research system, using an orchestrator-worker pattern, reports roughly how much more token cost than a single-agent chat interaction, and under what condition does that multiplier actually pay off?

- **A.** About 2x, and it pays off on any task that feels slow
- **B.** About 15x, and it pays off only when the task genuinely decomposes into independent parts explorable in parallel — not on tightly-coupled work like coding, where subagents mostly wait on each other
- **C.** About 15x, and it always pays off regardless of task structure
- **D.** Cost is identical to a single agent; only latency differs

> **Answer: B.** The ~15x figure is real and reported, but it only buys something on genuinely parallel-decomposable work. Fanning a tightly-coupled task out across subagents pays the multiplier without the parallel benefit — the standard incident pattern is tripling a bill for barely-changed output quality.

46. A business stakeholder says: "we need the agent to be fast and accurate." A developer is asked to turn this into a functional requirement suitable for an eval. Which of the following is a properly-formed functional requirement derived from it?

- **A.** "The agent should be fast and accurate" (restated as-is)
- **B.** "The system must be built using an approved cloud provider"
- **C.** "The agent produces a two-sentence summary naming the issue and current status, checkable against a reference answer"
- **D.** "Transcript data must not leave the EU"

> **Answer: C.** A functional requirement must be specific enough to check — "fast and accurate" is a business goal, not a requirement; "must use an approved cloud provider" and "data must not leave the EU" are infrastructure requirements, not functional ones.

47. A team is scoping a deployment for a customer that already holds SOC 2 and a specific cloud compliance certification, but hasn't stated a residency requirement yet. What is the correct sequencing per the systems-lifecycle "gate" discipline?

- **A.** Build the full integration first, then ask about compliance requirements at the security review
- **B.** Capture the residency and compliance requirements during the requirements phase, and gate the move from design to build on the chosen platform satisfying them
- **C.** Compliance requirements only matter at the deploy phase, not before
- **D.** Skip formal requirements gathering since the customer already has SOC 2

> **Answer: B.** Gates exist precisely to prevent discovering a residency/compliance mismatch after a build is complete — the requirements phase is where this constraint should surface, gating progression into design/build.

48. A regulated EU customer requires that model inference never occur outside the EU. Which statement about deployment platform choice is correct?

- **A.** The first-party Claude API supports EU-only data residency directly
- **B.** The direct Anthropic API does not currently provide EU data residency — route through Amazon Bedrock or Google Vertex AI with the region pinned to a covered jurisdiction instead
- **C.** Any deployment platform automatically satisfies EU residency once encryption is enabled
- **D.** EU residency is only a concern for FedRAMP workloads, not GDPR ones

> **Answer: B.** This is a specific, testable gotcha: the first-party API is not currently EU-resident. EU data-residency requirements route through a cloud-mediated platform with region explicitly pinned in the client configuration.

49. A support-ticket classifier is graded with an LLM-as-judge that returns only a numeric score (no reasoning). Scores cluster tightly around 6 regardless of actual output quality. What is the most likely cause, and the fix?

- **A.** The judge model is too weak — switch to a larger judge model
- **B.** The judge was never asked to produce reasoning before the score, so it drifted to a safe middle value; ask for strengths/weaknesses/reasoning first, then calibrate against human-labeled cases
- **C.** This is expected judge behavior and doesn't need fixing

- **D.** Exact-match grading should replace the judge entirely

> **Answer: B.** Reasoning-first is what anchors an LLM judge to something specific about the actual output; without it, judges commonly drift toward an uncommitted middle score. The second required step is calibrating measured agreement against human labels before trusting the result.

50. A feature passed all validation checks in production for two weeks, then extracted the wrong value from an input containing two dates. Validation confirmed the extracted date was well-formed and the field was populated. What does this reveal about the relationship between validation and evals?

- **A.** Validation and evals are redundant — one makes the other unnecessary
- **B.** Validation confirms a value is the *right shape*; it cannot confirm it's the *right value* — that requires a graded case with a human-checked expected output for that specific input pattern
- **C.** The bug was in the model, not in the test coverage
- **D.** This failure mode is not preventable by any testing method

> **Answer: B.** This is the exact incident pattern: shape-correctness (validation) and value-correctness (eval-graded expected behavior) are different guarantees. The fix was adding the two-date case as a graded example, not tightening validation rules.

51. An MCP server exposes ten tools. A team wants Claude to be able to call one specific read tool freely, while every other tool on that server — including a delete operation — still requires approval. What's the correct permission configuration?

- **A.** This isn't possible; MCP permissions apply at the whole-server level only
- **B.** An allow rule targeting `mcp__servername__specific_tool`, leaving the server's other tools ungated by that rule so they still prompt
- **C.** Disconnect the server and rebuild the delete tool as a custom tool instead
- **D.** Set `disable_parallel_tool_use` on the server

> **Answer: B.** MCP permission rules can target one tool specifically via `mcp__server__tool` syntax — an allow rule on one tool doesn't extend to the rest of the server's tools, which continue to follow whatever rule (or lack of one) otherwise applies to them.

52. Two tools, `search_docs` ("use this to find information about the product") and `get_context_summary` ("use this to retrieve relevant information from the current session"), cause Claude to repeatedly call the wrong one on ambiguous inputs. What's the correct fix, and what's the wrong fix?

- **A.** Correct: rename the tools to be more distinct. Wrong: touch the descriptions.
- **B.** Correct: add one exclusion sentence to each description naming when *not* to call it. Wrong: assume the tool names alone will eventually disambiguate given enough test runs.
- **C.** Correct: increase `max_tokens`. Wrong: modify the schema.
- **D.** Correct: switch to a larger model. Wrong: edit any prompt or schema content.

> **Answer: B.** Claude routes primarily on description content; both descriptions here reduce to "find information" with no distinguishing signal. Adding an explicit exclusion condition to each tool's description is the fix that generalizes — renaming alone doesn't change what Claude actually reads to decide.

53. A pipeline needs to send the same reference product image with every request across thousands of pipeline runs. Which image-encoding approach is correct, and why?

- **A.** Inline base64 every time — simplest to implement
- **B.** A public URL reference, since no payload travels with the request

- **C.** Files API — upload once, reference by `file_id`; overhead drops to near-zero after the first upload, versus re-paying the full payload cost on every run with base64
- **D.** It doesn't matter; all three methods have identical cost at scale

> **Answer: C.** This is exactly the reuse case the Files API exists for — a one-time upload cost versus base64's full-payload cost repeated on every one of thousands of runs.

54. A nightly classification job has been re-chunked into smaller batches three times and still hits rate limits every night. The developer is looping over the chunked list, calling the synchronous API once per item. What is the correct diagnosis?

- **A.** The chunks are still too large; chunk further
- **B.** This was never batching — looping over the synchronous endpoint produces one API call per item regardless of chunk size, hitting the same rate limits; switch to the Message Batches API, a genuinely different submission model
- **C.** Rate limits are fixed per account and cannot be worked around
- **D.** The fix is to add exponential backoff between chunks, not to change the submission method

> **Answer: B.** Chunking a list fed into a loop over the synchronous API doesn't change what the API sees — still one request per item. Only an actual Batch API submission (single call, `batch_id`, poll for completion) changes the request pattern and unlocks the lower per-token cost.

55. A firm handling attorney-client privileged documents wants to use Claude in their internal application. Which configuration is most likely to survive a legal/security review?

- **A.** Direct API calls from a consumer-grade Claude interface, since the firm trusts its own employees
- **B.** Direct API or SDK calls from inside the firm's own application, authenticated via SSO, routed through a firm-approved LLM gateway with full request/response logging implemented at the application layer
- **C.** Any configuration is acceptable as long as encryption is enabled in transit
- **D.** Privileged content should never be sent to any API under any configuration

> **Answer: B.** Consumer-grade surfaces aren't auditable end-to-end. Anthropic doesn't capture conversation content by default on direct API traffic, so the firm must implement its own logging at the application layer as part of an auditable, approved path.

56. A team building a government-facing application on AWS is told the workload requires FedRAMP High authorization. Which deployment option is authorized for this?

- **A.** Claude Enterprise via the AWS Marketplace listing
- **B.** Any Bedrock integration, since Bedrock itself is generically FedRAMP-authorized regardless of which Claude access pattern is used
- **C.** Claude via Amazon Bedrock GovCloud specifically
- **D.** The first-party Claude API with a signed BAA

> **Answer: C.** Bedrock GovCloud is one of the specific authorized FedRAMP High / DoD IL4/5 routes. Claude Enterprise on the general AWS Marketplace is explicitly *not* FedRAMP authorized — the distinction between GovCloud and standard Bedrock/Marketplace matters here.

57. During a code review, a reviewer flags that a PR only shows application logic diffs, with no mention of the tool schema changes bundled in the same PR. Why does this matter specifically for Claude-application code review?

- **A.** It doesn't — schema changes are internal implementation detail

- **B.** A tool schema change is a breaking-change risk for every caller of that tool, the same way a public API's breaking change would be, but nothing enforces catching it the way a compiler enforces a typed API contract
- **C.** Schema changes are automatically validated by the Messages API before merge
- **D.** Only prompt changes need review; schemas are statically typed and self-validating

> **Answer: B.** Without a type system enforcing the tool's contract, an unreviewed schema change is exactly the kind of silent breaking change that ships undetected — review scope needs to explicitly include schema and prompt diffs, not just surrounding logic.

58. A team packaged an agent template quickly under deadline pressure, hardcoding the customer's repo path and a few prompt fragments directly into the loop. Months later, a second team tries to reuse it for a similar engagement and cannot configure it at all. What was missing from the original build?

- **A.** The agent should have used a larger model
- **B.** Parameterized configuration for customer-specific values, documentation of assumptions, and a bundled eval proving the asset still works in a new context — a working build and a *reusable* build are different finishing states
- **C.** Nothing was missing; reuse failures are simply a normal part of software maintenance
- **D.** A more detailed system prompt would have solved the reuse problem

> **Answer: B.** A build that runs is not automatically packaged for reuse. The absence of parameters, documented assumptions, and a bundled eval are the three specific warning signs — and the cost of not packaging doesn't surface until someone else tries to reuse the build, by which point the original context is expensive to reconstruct.